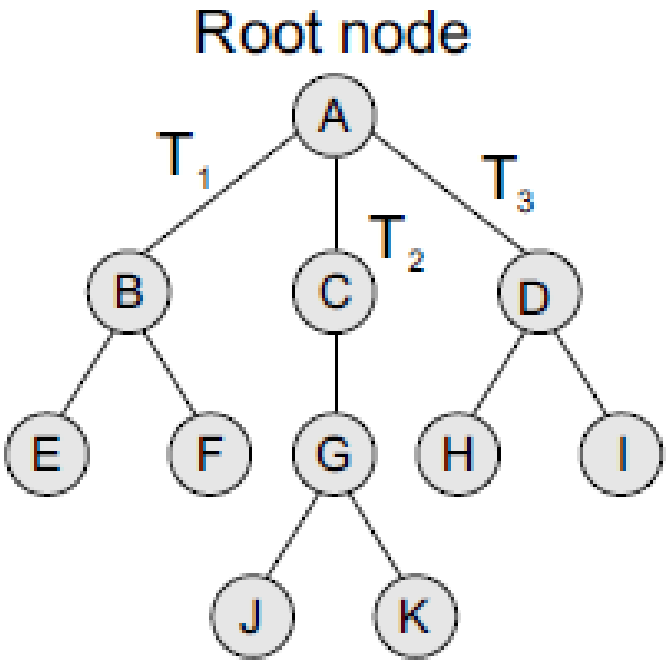
Tree

INTRODUCTION

- A tree is recursively defined as a set of one or more nodes where one node is designated as the root of the tree and all the remaining nodes can be partitioned into non-empty sets each of which is a sub-tree of the root.

Figure shows a tree where
node A is the root node;
nodes B, C, and D are children of the root node and form sub-trees of the tree rooted at node A.

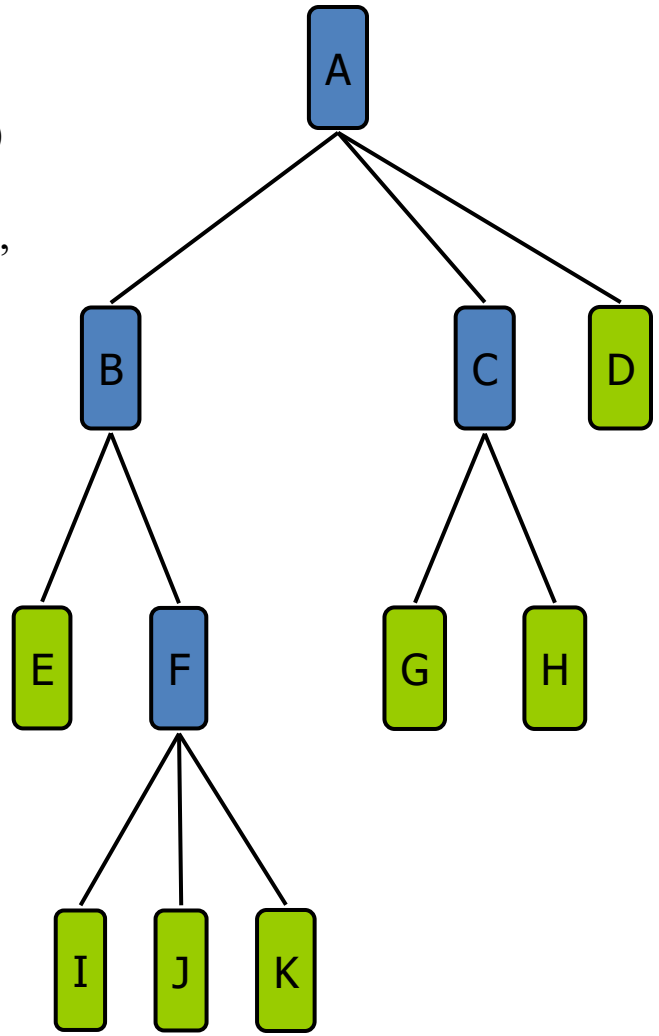


Uses

1. Stores natural hierarchical data (file system)
2. Help to organize data and perform efficient operation (search, sort, eg BST)
3. Used to design and implement Network routing algorithm
4. Represent and store algebraic expression

Tree Terminology

- **Root**: node without parent (A)
- **Siblings**: nodes share the same parent
- **Internal node**: node with at least one child (A, B, C, F)
- **External node** (leaf): node without children (E, I, J, K, G, H, D)
- **Ancestors** of a node: parent, grandparent, grand-grandparent, etc.
- **Descendant** of a node: child, grandchild, grand-grandchild, etc.
- **Depth** of a node: number of ancestors
- **Height** of a tree: maximum depth of any node
- **Degree** of a node: the number of its children
  The leaf of the **tree** does not have any child so its **degree** is zero
- **Degree** of a tree: the maximum degree of a node in the tree.
- **Subtree**: tree consisting of a node and its descendants
- Empty (**Null**)-**tree**: a **tree** without any node.
- Root-**tree**: a **tree** with only one node

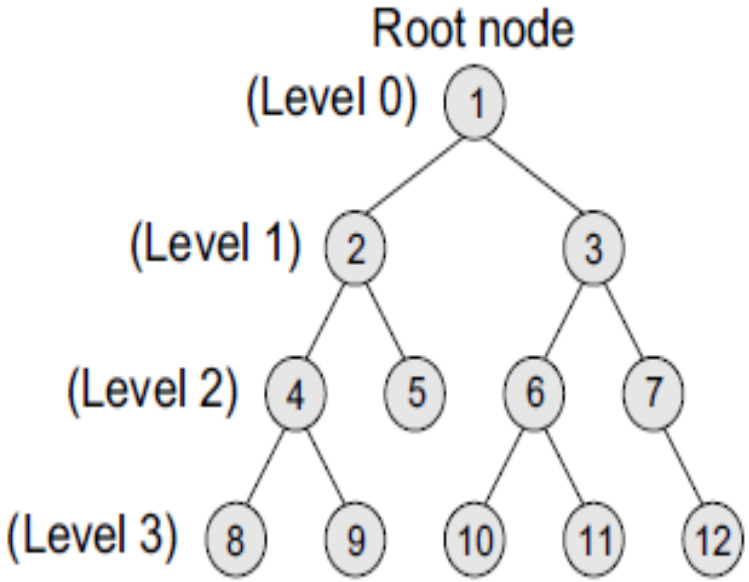


Binary Trees

It is a data structure that is defined as a collection of elements called nodes.

In a binary tree,

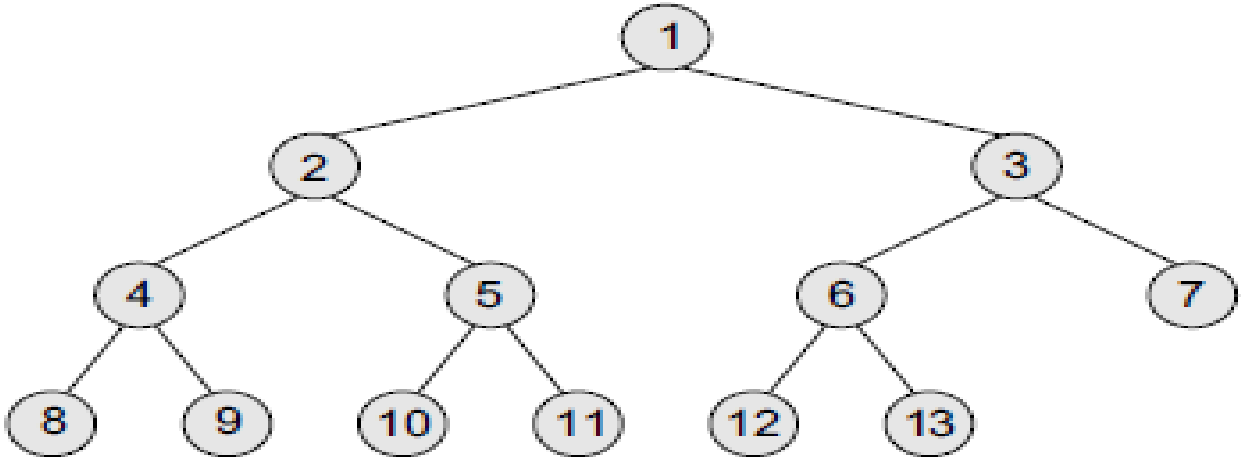
- the topmost element is called the root node, and
- Each node has 0, 1, or at the most 2 children.
- A node that has zero children is called a leaf node or a terminal node.
- Every node contains a **data element**, a **left pointer** which points to the left child, and a **right pointer** which points to the right child.



Complete Binary Trees

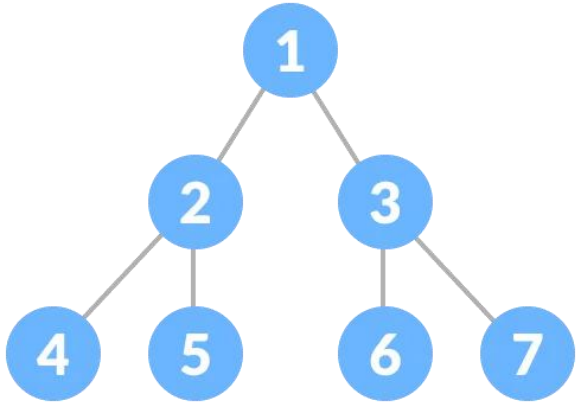
A *complete binary tree* is a binary tree that satisfies two properties.

- First, in a complete binary tree, every level, except possibly the last, is completely filled.
- Second, all nodes appear as far left as possible.



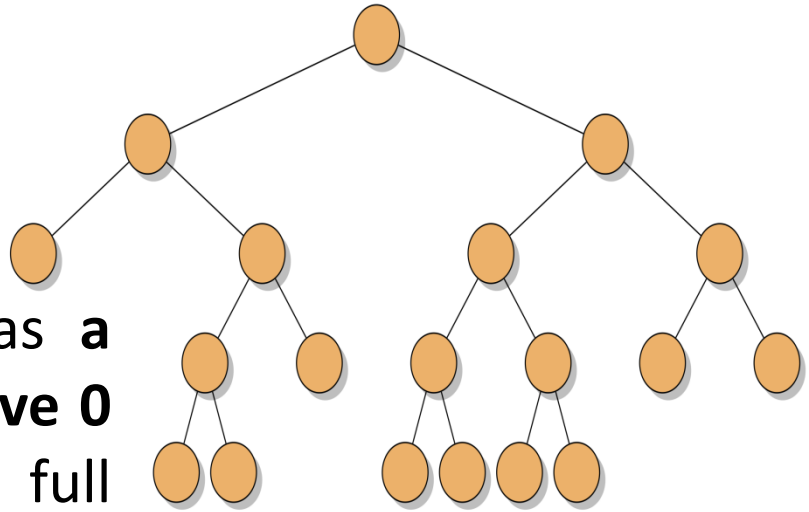
Perfect binary tree

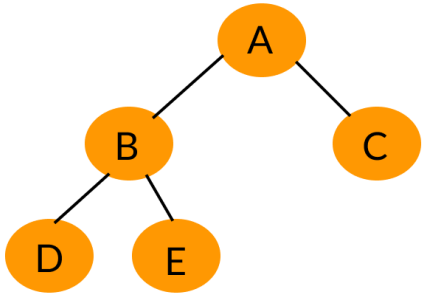
A perfect binary tree is **a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level**

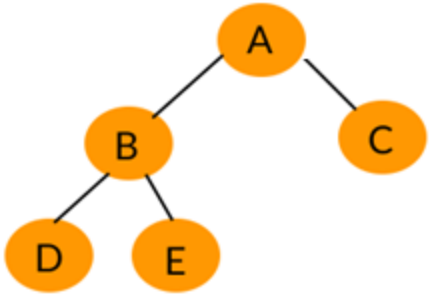


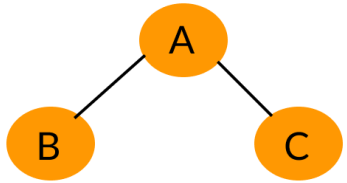
Full binary tree

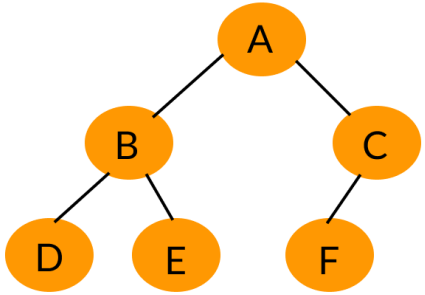
A full binary tree can be defined as **a binary tree in which all the nodes have 0 or two children**. In other words, the full binary tree can be defined as a binary tree in which all the nodes have two children except the leaf nodes

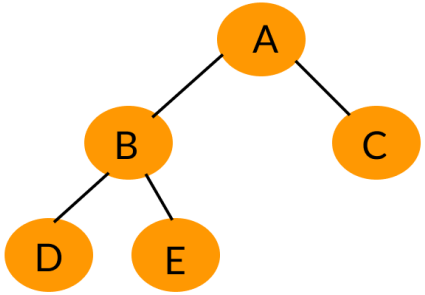


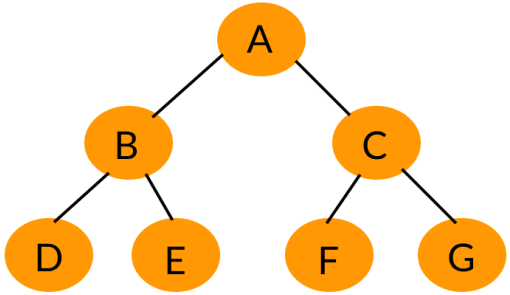












Complete Binary Tree

Full Binary Tree

Perfect Binary Tree

Representation of Binary Trees

Representation of Binary Trees in the Memory

Linked representation of binary trees

In the linked representation, every node will have three parts:

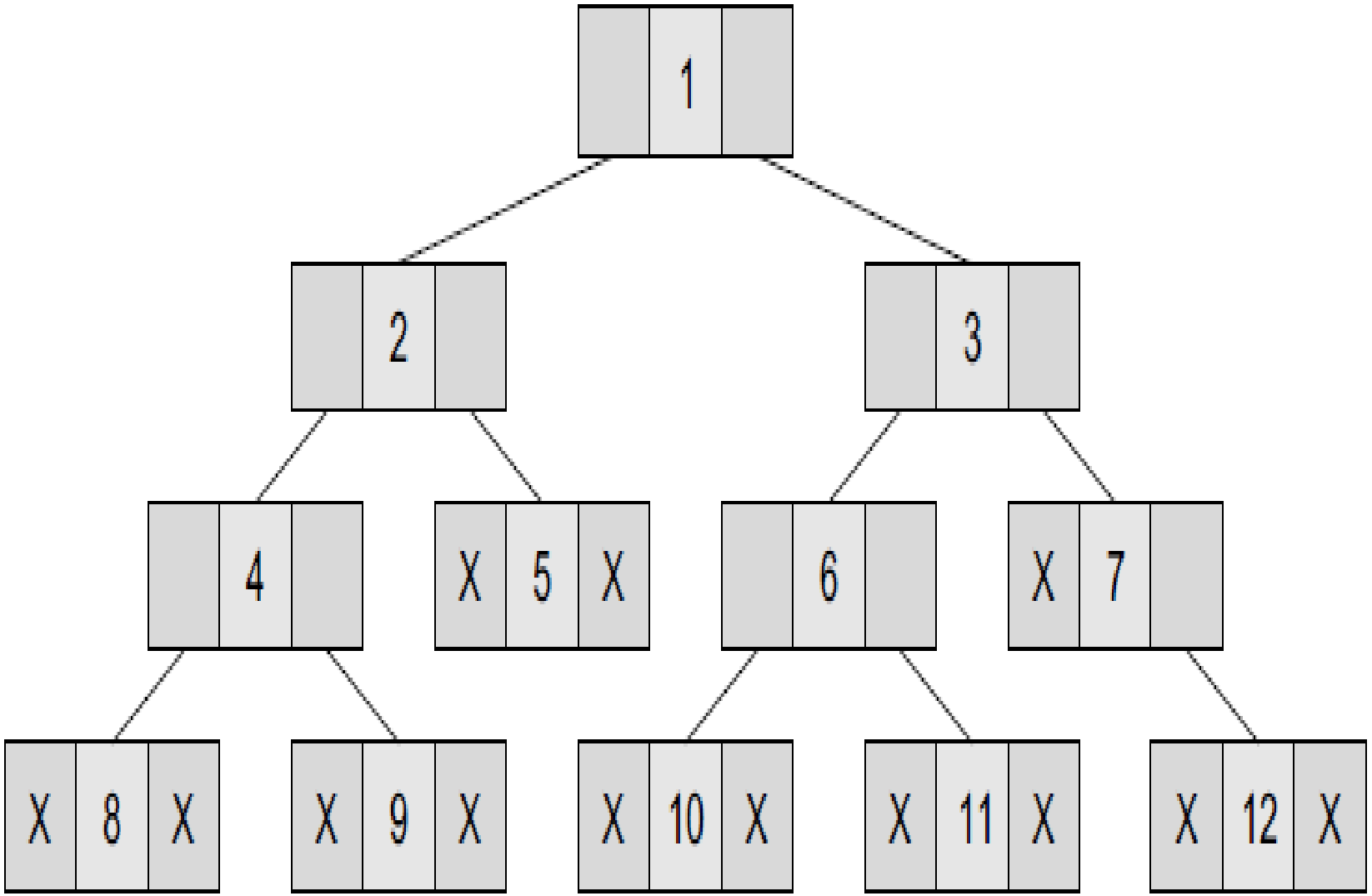
the **data element**, a **pointer to the left node**, and a **pointer to the right node**.

So in C++, the binary tree is built with a node type given below.

```
class node {
node *left;
int data;
node *right;
};
```

Every binary tree has a **pointer ROOT**, which points to the root element (topmost element) of the tree. If **ROOT = NULL**, then the tree is empty.

Linked representation of binary trees



Sequential representation of binary tree

Sequential representation of trees is done using single or one-dimensional arrays. Though it is the simplest technique for memory representation**, it is inefficient** as it requires a lot of memory space.

A sequential binary tree follows the following rules:

1. A one-dimensional array, called TREE, is used to store the elements of tree.

2. The root of the tree will be stored in the first location. That is, TREE[1] will store the data of the root element.

3. The children of a node stored in location K will be stored in locations $(2 \times K)$ and $(2 \times K+1)$.

4. The maximum size of the array TREE is given as $(2^{h+1}–1)$, where h is the height of the tree.

5. An empty tree or sub-tree is specified using NULL. If TREE[1] = NULL, then the tree is empty.

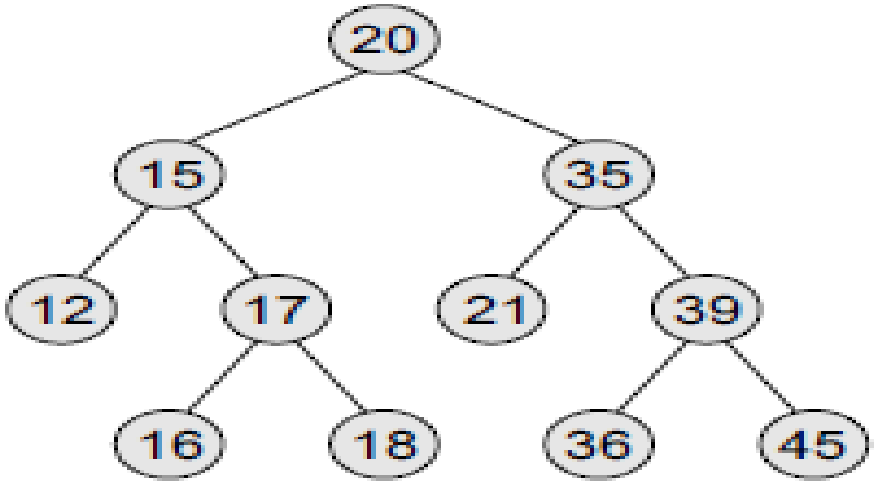
Array Implementation of a binary tree

For any node k

The left child of a node K = **2 × K**

The right child of a node K = **2 × K+1**

Parent of any node K = **floor (K/2)**



1	20
2	15
3	35
4	12
5	17
6	21
7	39
8	
9	
10	16
11	18
12	
13	
14	36
15	45

Traversing a Binary Tree

- Traversing a binary tree is the process of visiting each node in the tree exactly once in a systematic way.

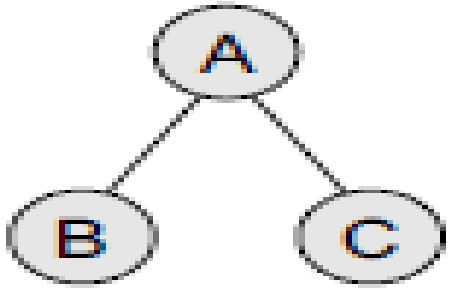
- Unlike linear data structures in which the elements are traversed sequentially, tree is a non linear data structure in which the elements can be traversed in many different ways. (ex. LNR, LRN, NLR)

- There are different algorithms for tree traversals. These algorithms differ in the order in which the nodes are visited.

Pre-order Traversal

To traverse a non-empty binary tree in pre-order, the following operations are performed recursively at each node. The algorithm works by:

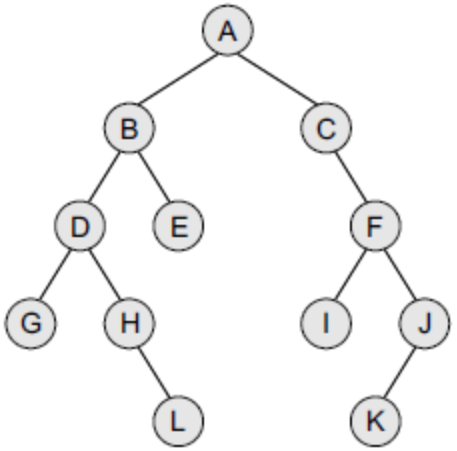
1. Visiting the root node,
2. Traversing the left sub-tree, and finally
3. Traversing the right sub-tree.



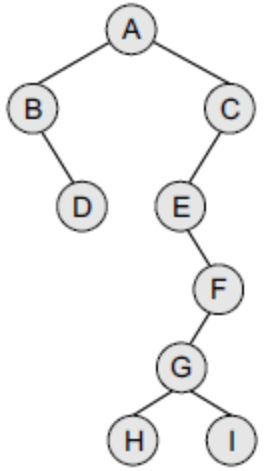
The pre-order traversal of the tree is given as A, B, C.

Algorithm for Pre-Order

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:          Write TREE->DATA
Step 3:          PREORDER(TREE->LEFT)
Step 4:          PREORDER(TREE->RIGHT)
      [END OF LOOP]
Step 5: END
```



(a)



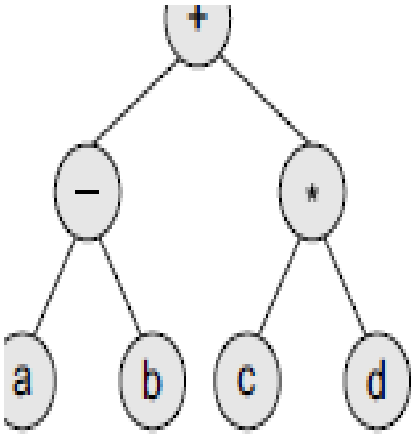
(b)

TRAVERSAL ORDER: A, B, D, G, H, L, E, C, F, I, J, K

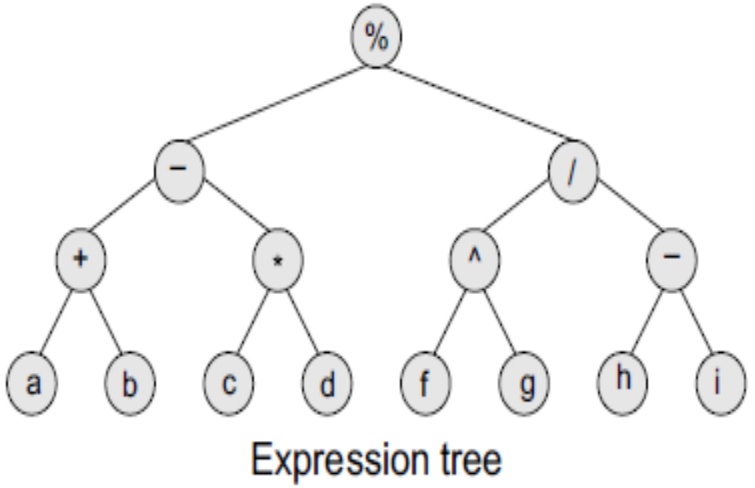
TRAVERSAL ORDER: A, B, D, C, E, F, G, H, I

Pre-order traversal algorithms are **used to extract a prefix notation** from an expression tree.

For example, consider the expressions given below. When we traverse the elements of a tree using the **pre-order traversal** algorithm, the expression that we get is a prefix expression.



+ – a b * c d



Expression tree

% – + a b * c d / ^ f g – h i

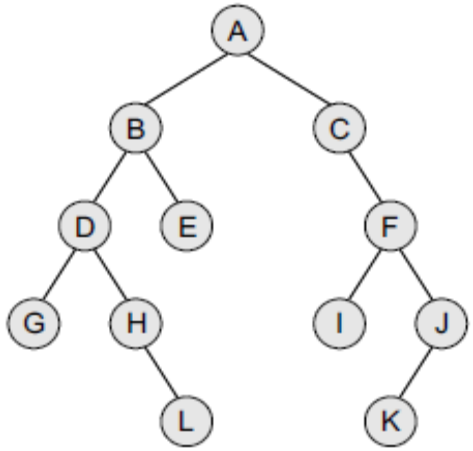
In-order Traversal

To traverse a non-empty binary tree in in-order, the following operations are performed recursively at each node. The algorithm works by:

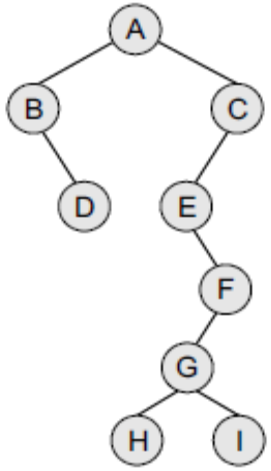
1. Traversing the left sub-tree,
2. Visiting the root node, and finally
3. Traversing the right sub-tree.

Algorithm for In-Order

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:          INORDER(TREE->LEFT)
Step 3:          Write TREE->DATA
Step 4:          INORDER(TREE->RIGHT)
      [END OF LOOP]
Step 5: END
```



(a)



(b)

TRAVERSAL ORDER: G, D, H, L, B, E, A, C, I, F, K, and J

TRAVERSAL ORDER: B, D, A, E, H, G, I, F, and C

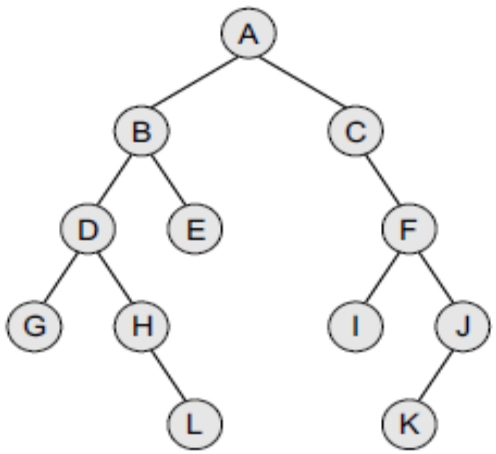
Post-Order Traversal

To traverse a non-empty binary tree in post-order, the following operations are performed recursively at each node. The algorithm works by:

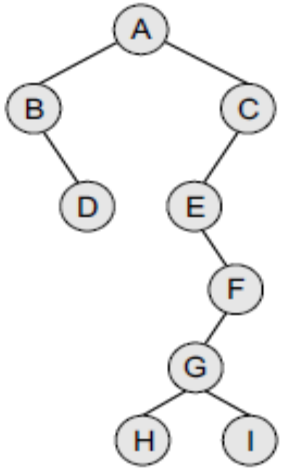
1. Traversing the left sub-tree,
2. Traversing the right sub-tree, and finally
3. Visiting the root node.

Algorithm for Post-Order

```
Step 1: Repeat Steps 2 to 4 while TREE != NULL
Step 2:          POSTORDER(TREE->LEFT)
Step 3:          POSTORDER(TREE->RIGHT)
Step 4:          Write TREE->DATA
       [END OF LOOP]
Step 5: END
```



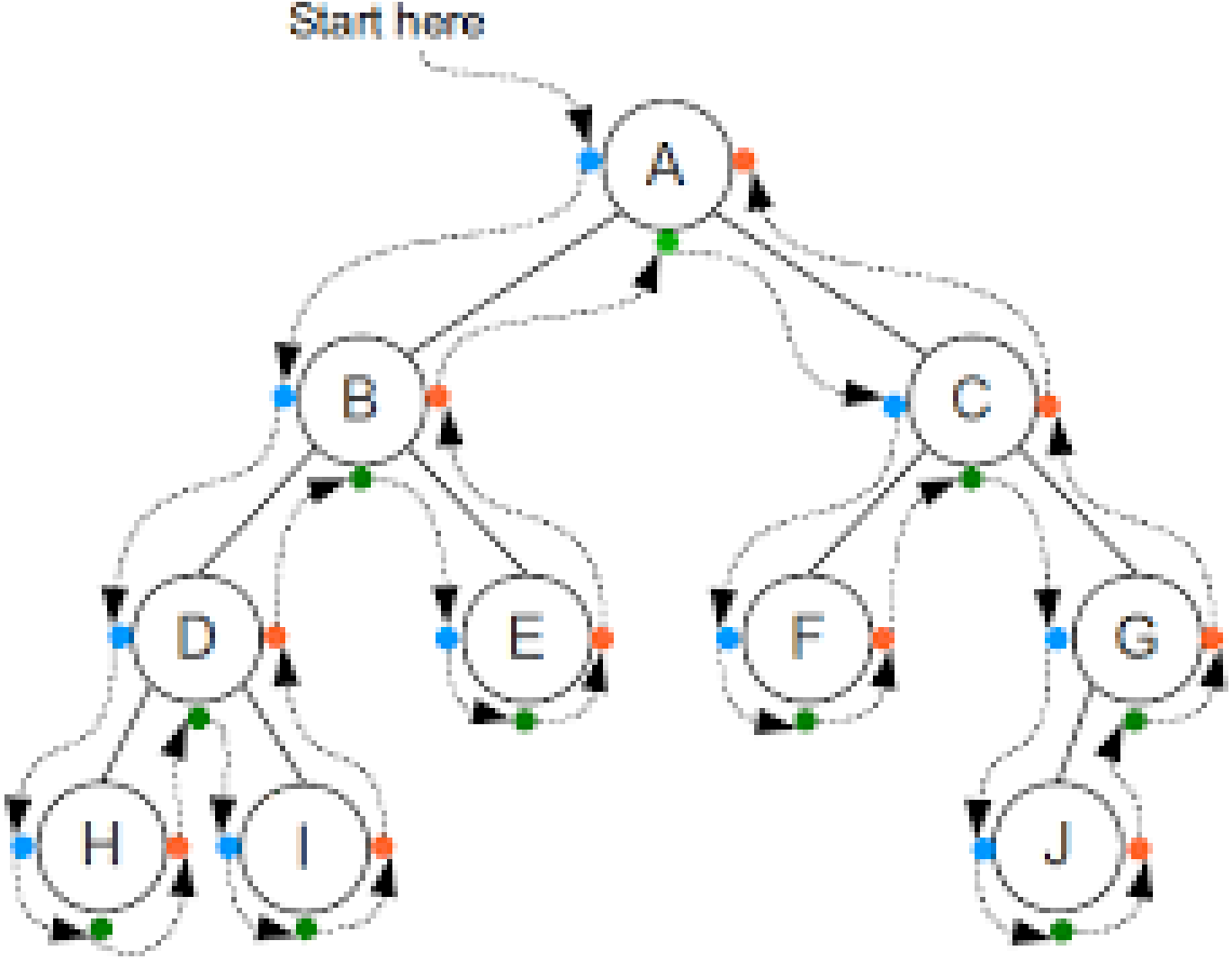
(a)



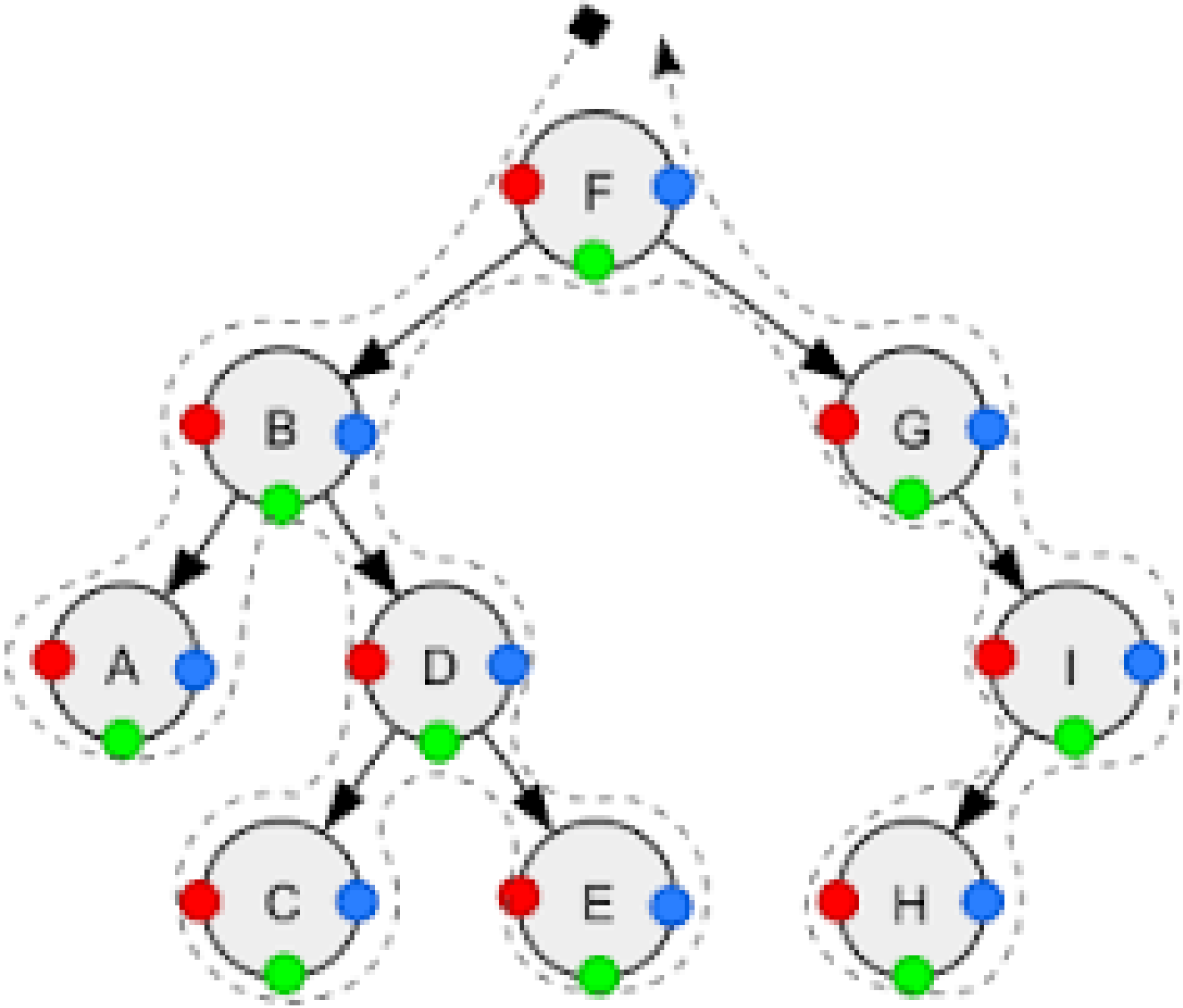
(b)

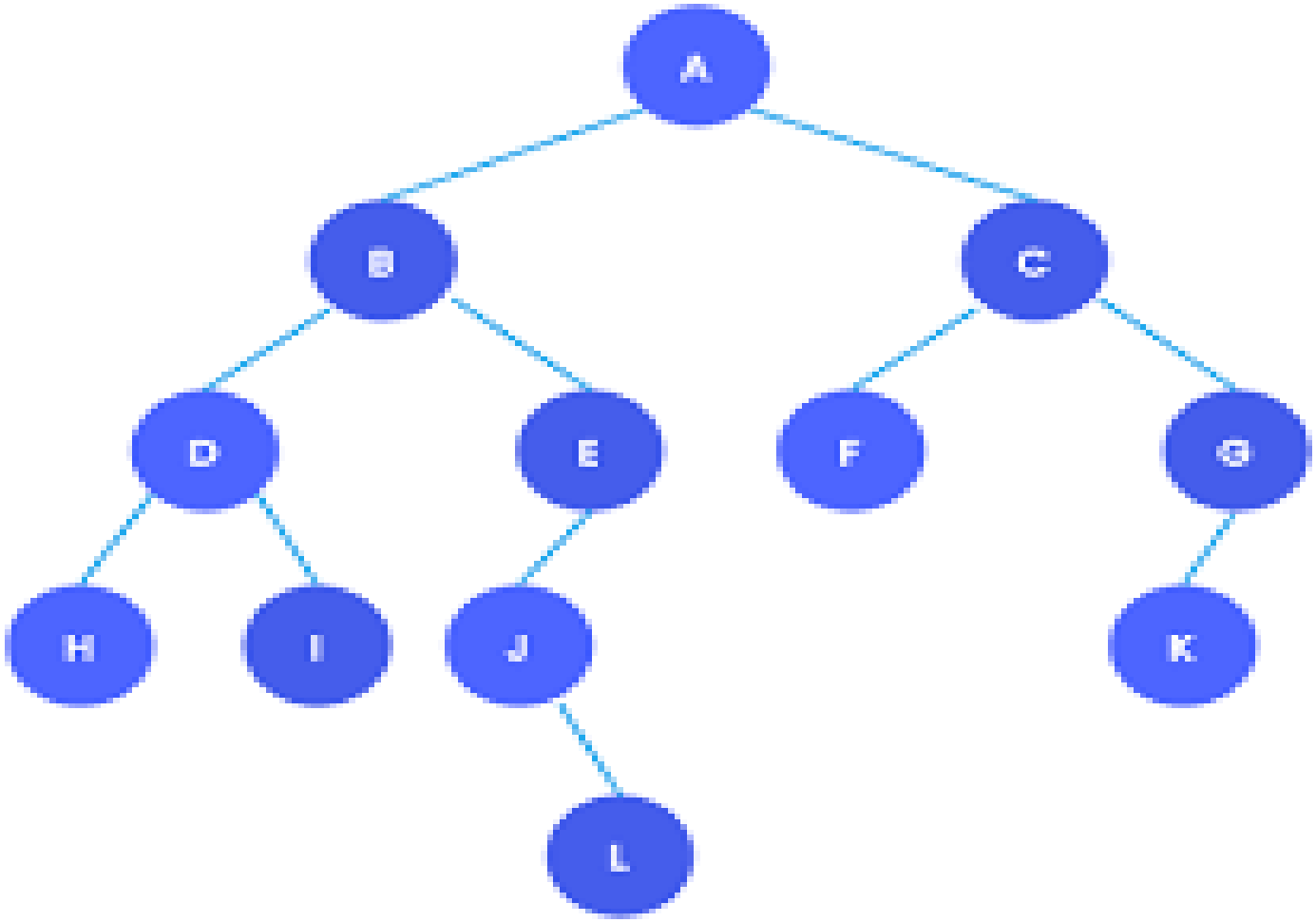
TRAVERSAL ORDER: G, L, H, D, E, B, I, K, J, F, C, and A

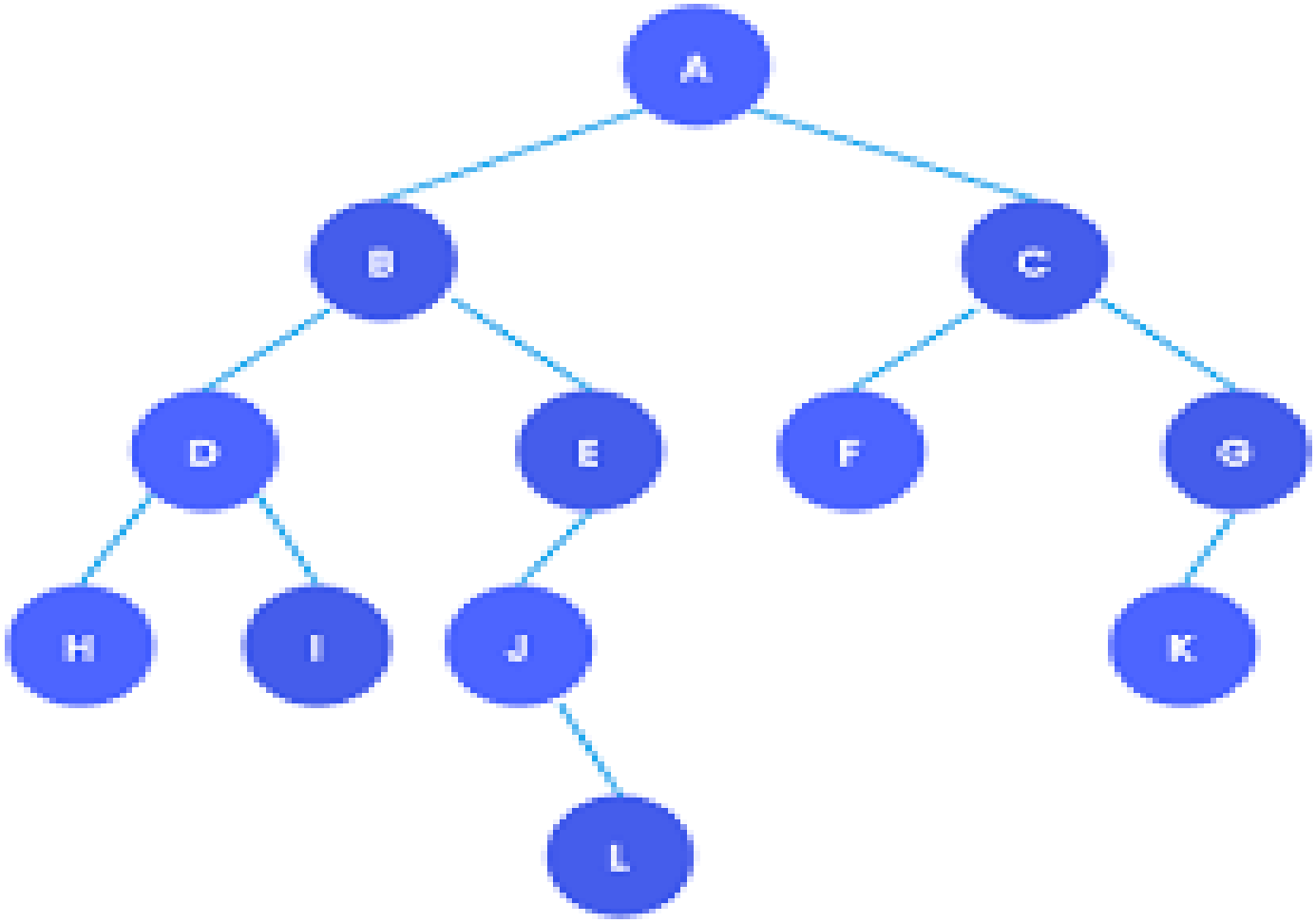
TRAVERSAL ORDER: D, B, H, I, G, F, E, C, and A



Pre-Order	ABDHIECFGJ
In-Order	HDIBEAFCJG
Post-Order	HIDEBFJGCA


F
B
G
A
D
I
C
E
H


A
B
C
D
E
F
G
H
I
J
K
L



In-order Traversal - H, D, I, B, J, L, E, A, F, C, K ,G

Pre-order Traversal - A, B, D, H, I, E, J, L, C, F, G, K

Post-order Traversal - H, I, D, L, J, E, B, F, K, G, C, A

Constructing Binary Tree Using Traversal Orders

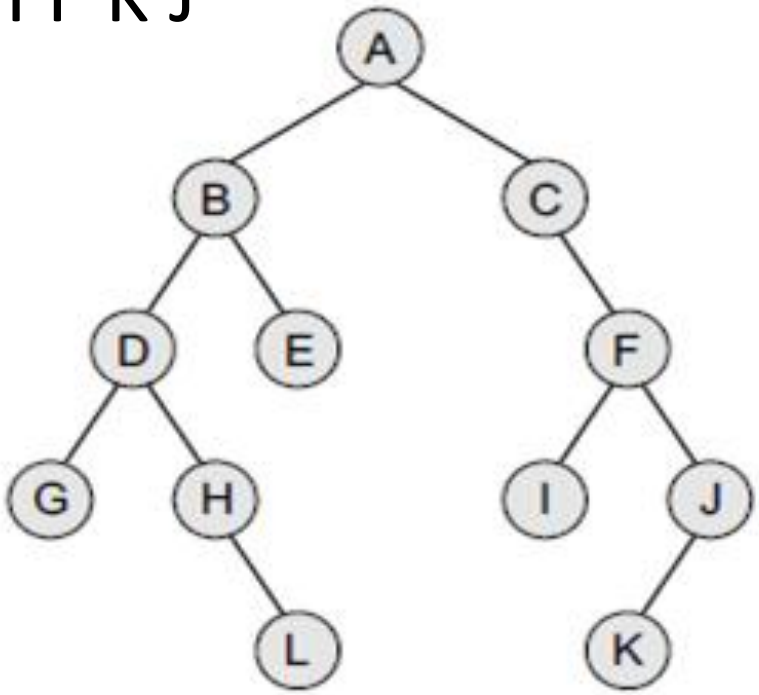
- Construct Tree
- Determine Postorder of following tree

=> Preorder – A B D G H L E C F I J K

=> Inorder - G D H L B E A C I F K J

- Postorder ?

- Preorder – A B D G H L E C F I J K
- Inorder - G D H L B E A C I F K J



- Postorder

G, L, H, D, E, B, I, K, J, F, C, and A

Constructing Binary Tree Using Traversal Orders

- Construct Tree
- Determine Preorder of following tree

=> Postorder – G L H D E B I K J F C A

=> Inorder - G D H L B E A C I F K J

- Preorder ?

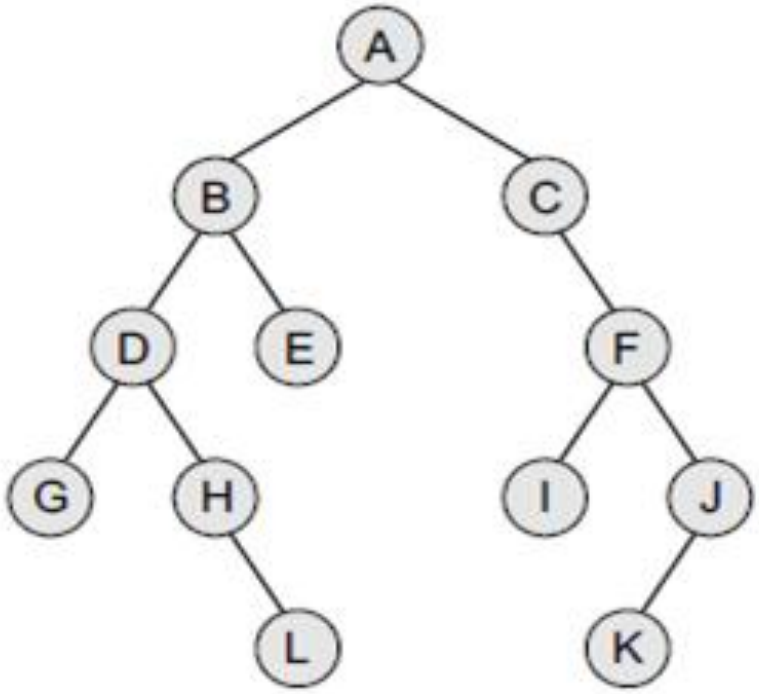
Constructing Binary Tree Using Traversal Orders

=> Postorder – G L H D E B I K J F C A

=> Inorder - G D H L B E A C I F K J

- Preorder ?

A B D G H L E C F I J K



Construct Tree using following Traversals

In-order Traversal - H, D, I, B, J, L, E, A, F, C, K ,G

Pre-order Traversal - A, B, D, H, I, E, J, L, C, F, G, K

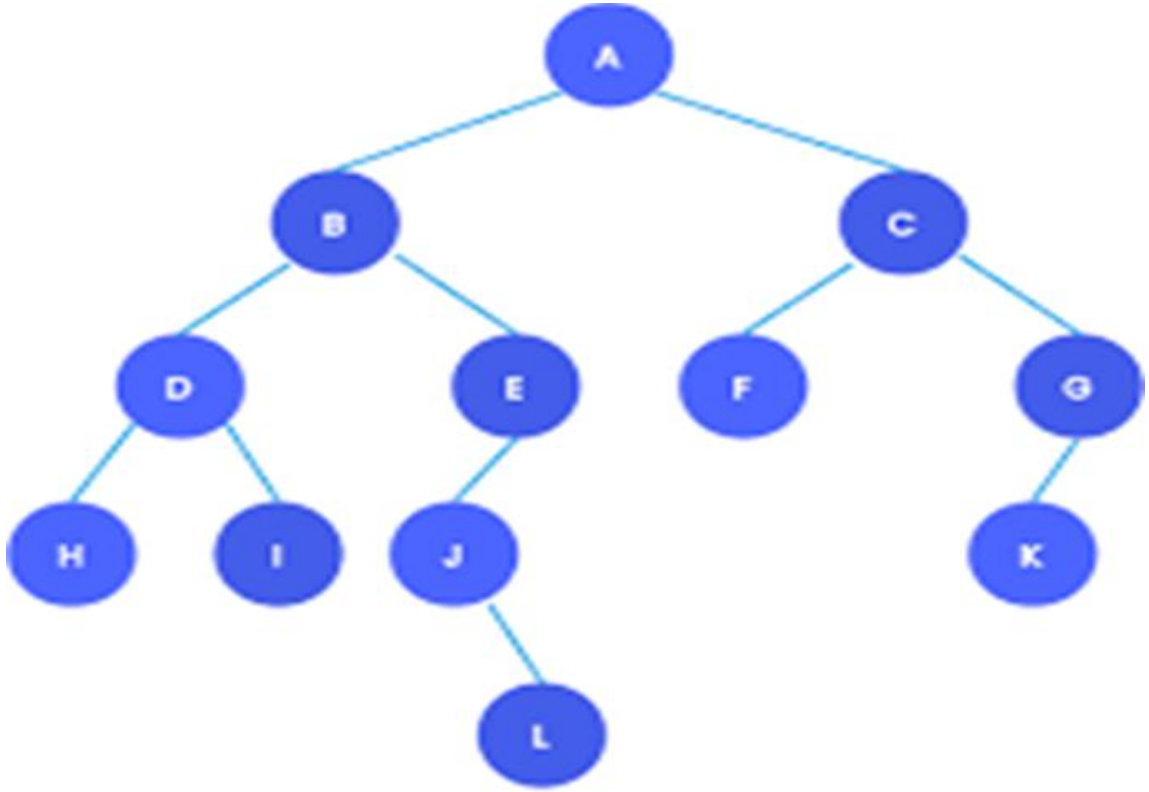
In-order Traversal - H, D, I, B, J, L, E, A, F, C, K ,G

Post-order Traversal - H, I, D, L, J, E, B, F, K, G, C, A

Construct Tree using following Traversals

In-order Traversal - H, D, I, B, J, L, E, A, F, C, K ,G

Pre-order Traversal - A, B, D, H, I, E, J, L, C, F, G, K



In-order Traversal - H, D, I, B, J, L, E, A, F, C, K ,G

Post-order Traversal - H, I, D, L, J, E, B, F, K, G, C, A

Constructing Binary Tree Using

PostOrder Traversal & PreOrder Traversal

Constructing Binary Tree Using Traversal Orders

=> Postorder – G L H D E B I K J F C A

$\Rightarrow$ Preorder - A B D G H L E C F I J K

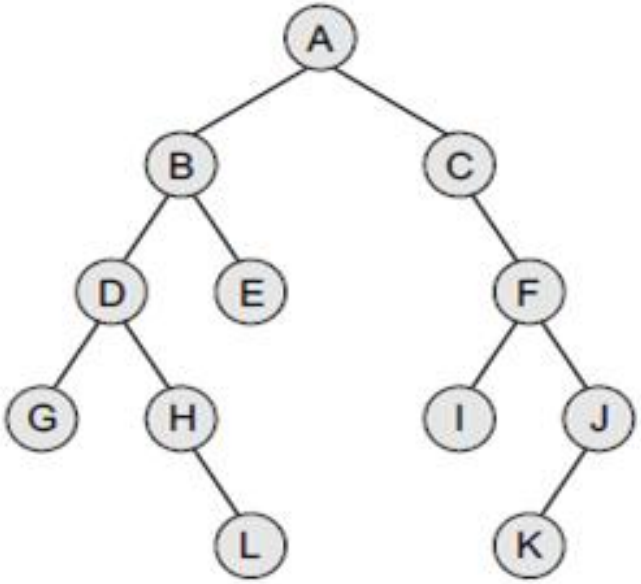
$\Rightarrow$Inorder ?

Constructing Binary Tree Using Traversal Orders

=> Postorder – G L H D E B I K J F C A

⇒ Preorder - A B D G H L E C F I J K

⇒Inorder ? G D H L B E A C I F K J

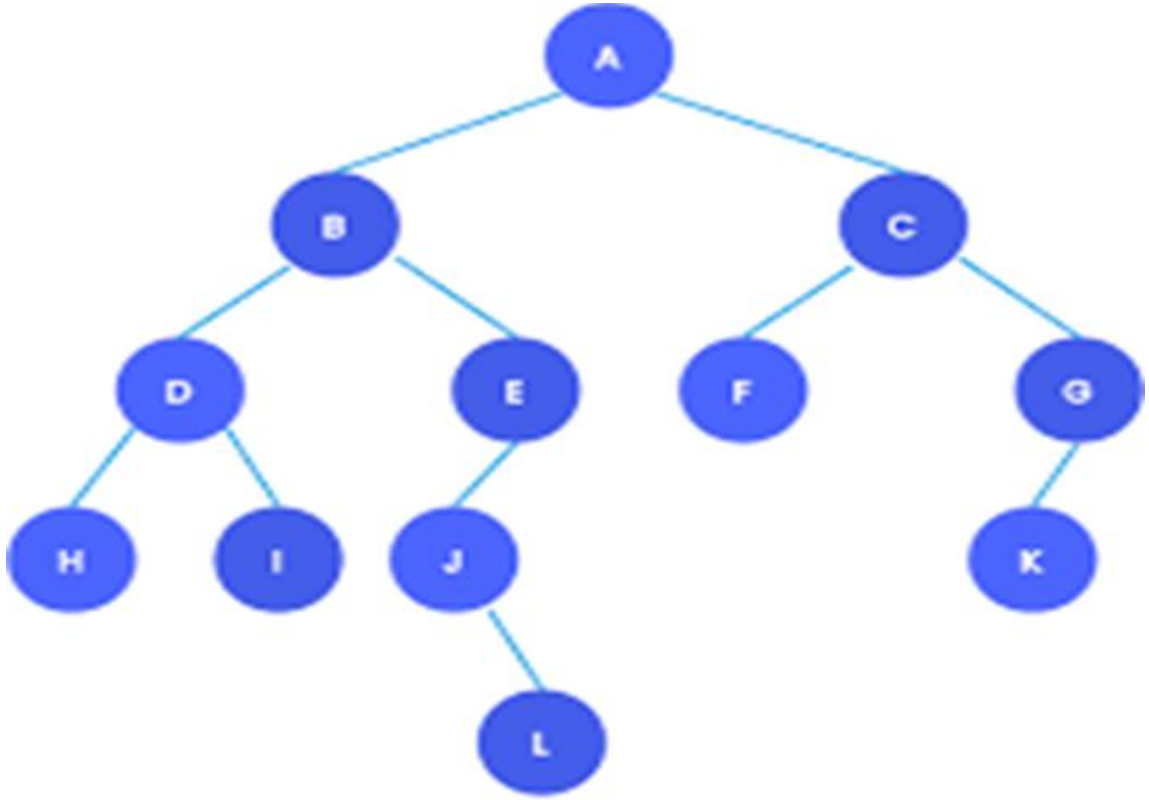


Pre-order Traversal - A, B, D, H, I, E, J, L, C, F, G, K

Post-order Traversal - H, I, D, L, J, E, B, F, K, G, C, A

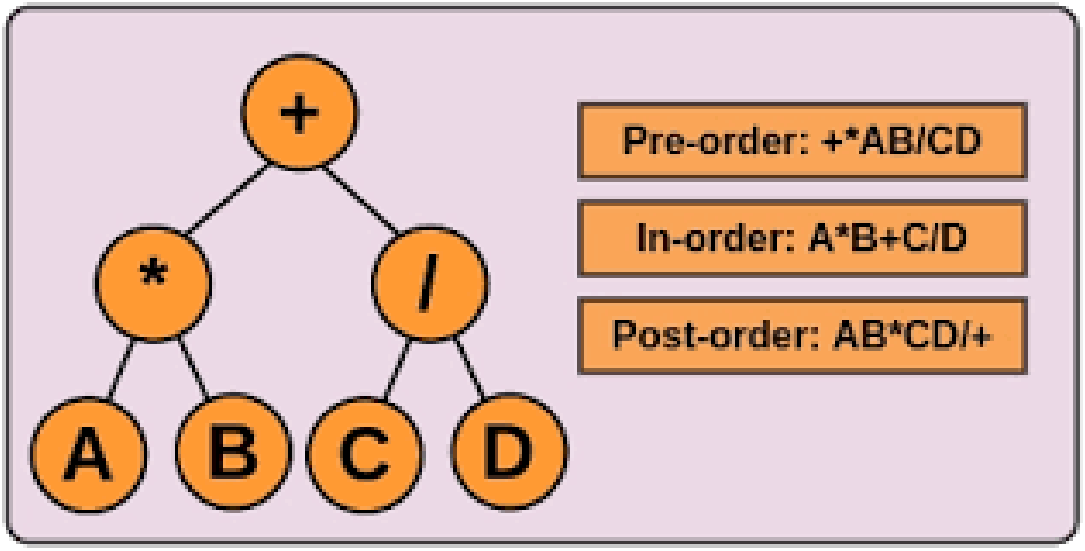
Pre-order Traversal - A, B, D, H, I, E, J, L, C, F, G, K

Post-order Traversal - H, I, D, L, J, E, B, F, K, G, C, A



Understanding

Expression Trees

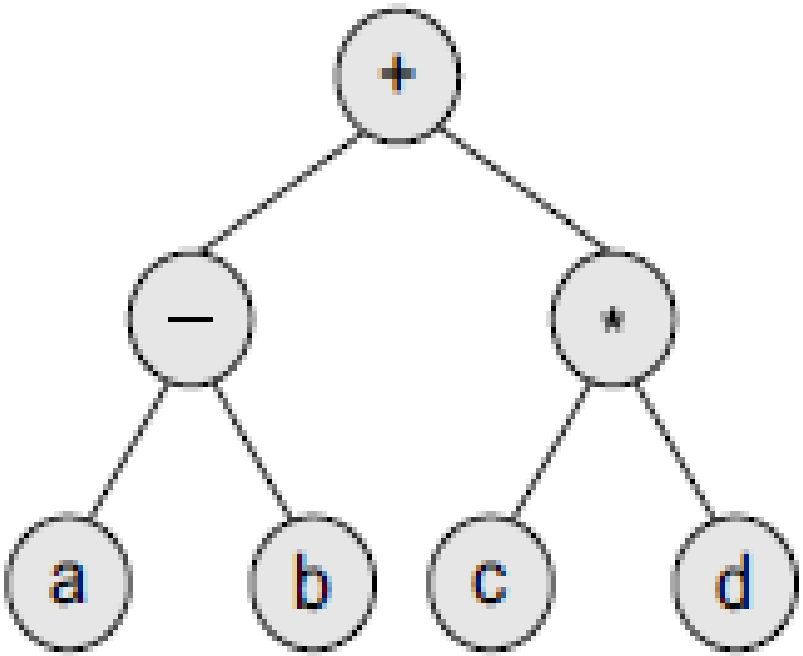


Expression Tree

Binary trees are widely used to store algebraic expressions.

For example, consider the algebraic expression given as:

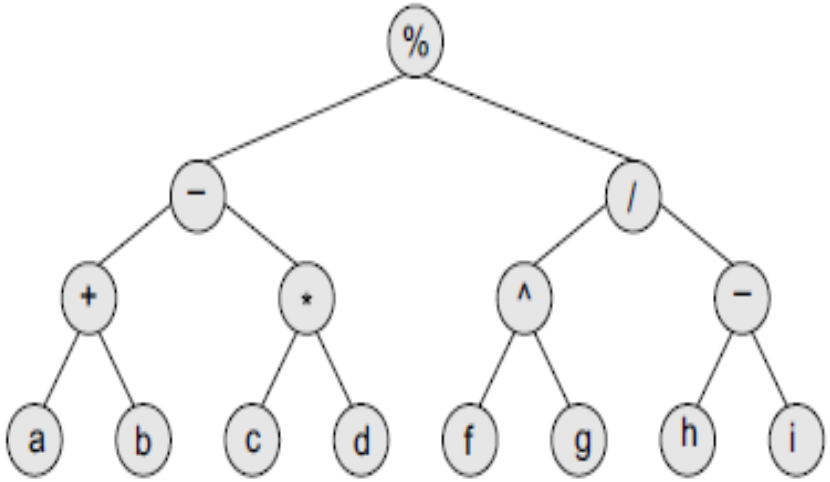
Exp = (a – b) + (c * d)



Given an expression,

Exp = **((a + b) – (c * d)) % ((f ^g) / (h – i)),**

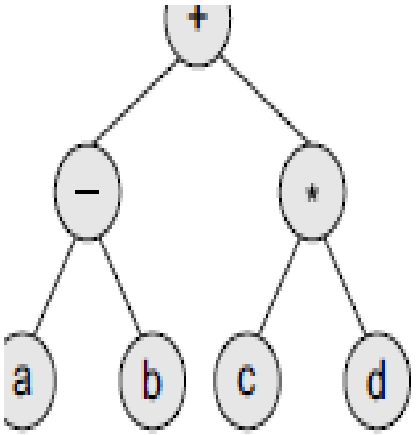
construct the corresponding binary tree.



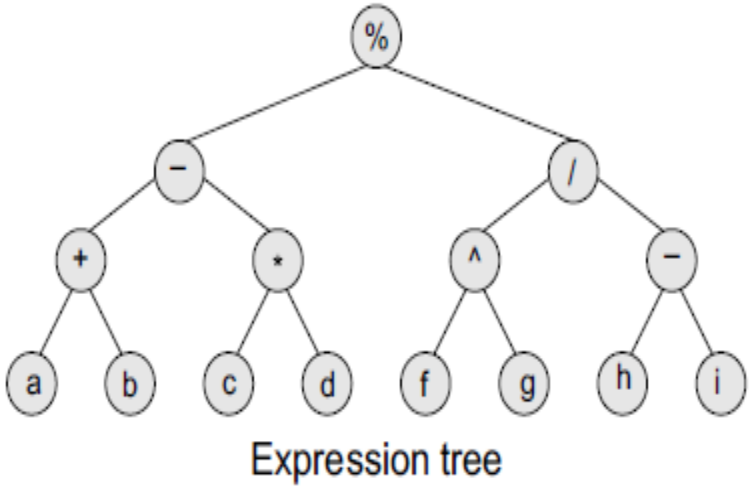
Expression tree

Pre-order traversal algorithms are **used to extract a prefix notation** from an expression tree.

For example, consider the expressions given below. When we traverse the elements of a tree using the **pre-order traversal** algorithm, the expression that we get is a prefix expression.



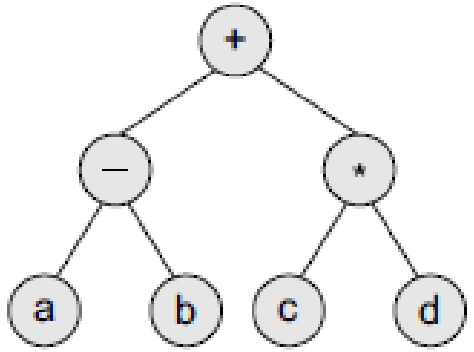
+ – a b * c d



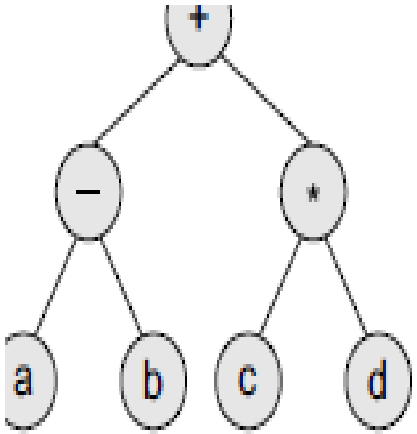
Expression tree

% – + a b * c d / ^ f g – h i

Significance of Prefix, Postfix and Infix Notation



Infix → (a – b) + (c * d)



Prefix → + – a b * c d

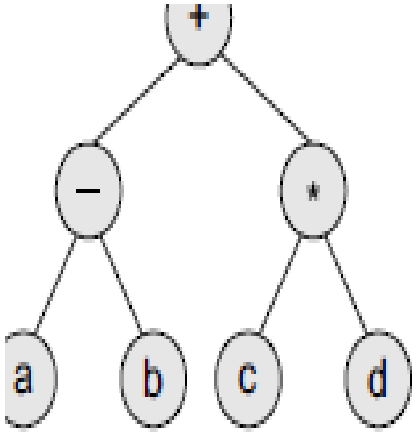
Construct **expression tree**

1. **a + b / c * d - e**
2. **a * b / c + d / e * f + g – h * i**
3. **(a + b) * (c * d - e) * f / g**

Construct Expression Tree

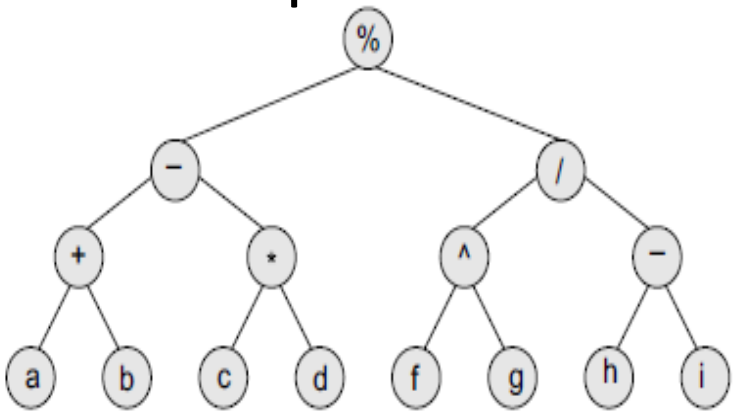
from

Prefix Notation



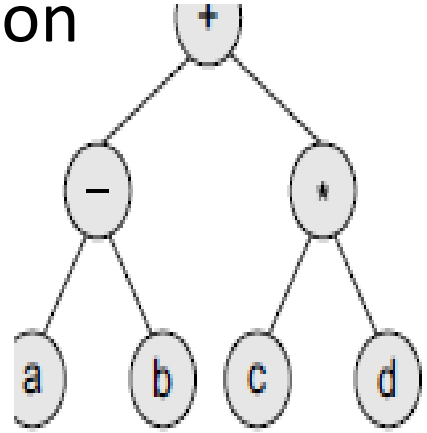
Prefix → + – a b * c d

Construct Expression tree from Prefix Notation



Expression tree

% – + a b * c d / ^ f g – h i

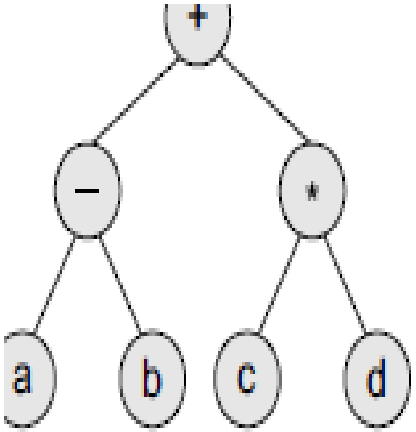


Prefix → + – a b * c d

Construct Expression Tree

from

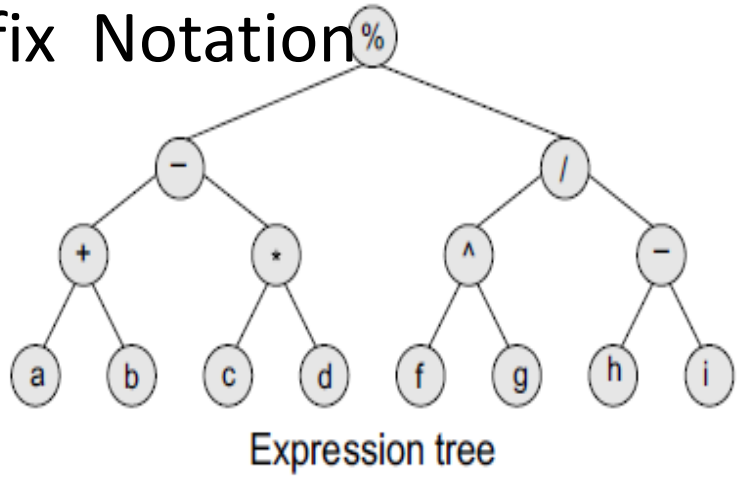
Postfix Notation



Postfix → a b - c d * +

Construct Expression tree from Postfix Notation

a b + c d * - f g ^ h i - / %



Expression tree